

Finding Regularity:

High-Performance Computing for Graph Algorithms

Gökhan Göktürk

High-performance computing and parallel algorithms

Computer Science Seminar

Research background

Sabancı University

BSc · MSc · PhD · PostDoc

Research

- High-performance computing
- Parallel and distributed algorithms
- GPU and multi-GPU computing
- Graph and probabilistic algorithms
- Performance engineering

Technical consulting

- Şişecam
- Akbank
- Sigorta Bilgi Merkezi
- Aksigorta

High-Performance Computing (HPC)

Designing **algorithms and systems** to solve demanding computational problems efficiently.

- **Reducing work:** exploiting mathematical structure, avoiding redundant computation, reusing intermediate results.
- **Changing representation:** sparse formats, compressed data, compact sketches.
- **Reformulating computation:** batching, fusion, blocking, and different execution orders.
- **Managing accuracy–cost tradeoffs:** approximation, sampling, or reduced precision when the application permits them.

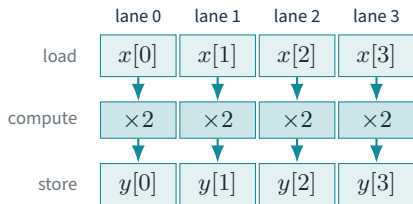
Hardware: multicore CPUs and vector units · GPUs and other accelerators · clusters

The goal: solve larger problems in less time with the available resources.

Why parallelism is not automatically performance

A **lane** is one position in a group performing an operation in parallel.

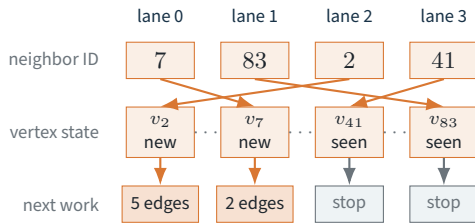
Regular array update: $y[i] = 2x[i]$



Predictable, direct access

Adjacent array elements; equal work per lane.

Graph traversal: data-dependent addresses



Unpredictable, indirect access

Scattered vertex state; unequal work per lane.

More lanes help when data access and work distribution keep them productive.

The research question

How can we reorganize irregular computation so that modern parallel hardware can execute it efficiently?

Regularity: predictable memory access and similar work across parallel lanes.

What can I change?

What can it improve?

Which work runs together

Reuse data; bring memory accesses closer together.

What state is stored

Reduce the memory footprint; make updates uniform.

Where work executes

Match work to CPU, GPU, and device-local memory.

Each change must preserve the required result, sampling distribution, or accuracy target.

Running example: influence maximization

Which k seeds maximize their combined influence?

Spread is stochastic; influence overlaps.

Select k seeds (priority queue)

Evaluate trial seed set S :

FOR each simulation $\rho = 1, \dots, R$

Sample edges using p_e .

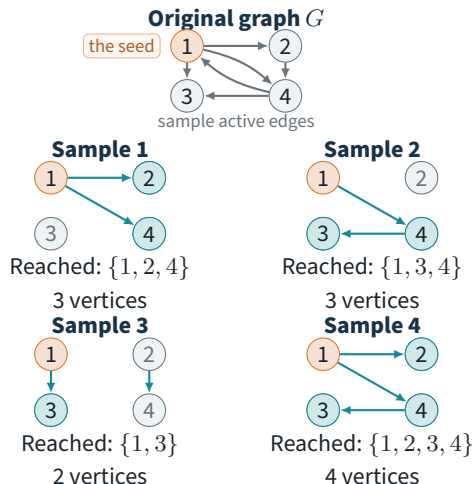
Traverse from S :

$\text{Reach}_{G_\rho}(S) = \{u \in V : \exists s \in S, s \rightsquigarrow_{G_\rho} u\}$

Count $|\text{Reach}_{G_\rho}(S)|$: distinct reachable vertices in sampled graph G_ρ .

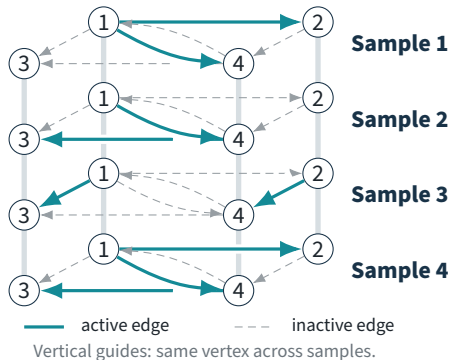
Average: $\hat{R}(S) = \frac{1}{R} \sum_{\rho=1}^R |\text{Reach}_{G_\rho}(S)|$

Repeated traversals: candidates $\times$ simulations.

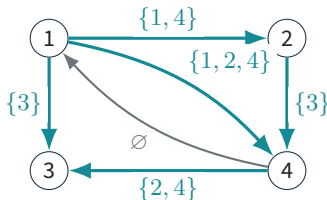


Where is the regularity hiding?

The same graph across samples



One topology, sample membership on edges



Shared across samples

topology · vertices · edges · algorithmic structure

Changes across samples

edge membership · active state · random decisions

Regularity comes from applying the same edge operation across samples.

Parallel lanes across samples

Vertex-major storage: each row holds the sample states for one vertex.

One sample; parallel neighbors

	r_0	r_1	r_2	r_3	
$v = 2$	lane 0				...
$v = 4$	lane 1				...
$v = 7$	lane 2				...
$v = 9$	lane 3				...

Different rows; scattered state accesses

4 random reads; likely 4 random writes

CPU: non-vector compute

GPU: warp/workgroup divergence and stalls

Independent thread scheduling helps;

scattered memory access remains expensive.

NVIDIA: 32-lane warps. AMD RDNA 3/4: 32/64-lane waves + VLIW-like dual issue.

AVX-512: 512 bits (16×32). Arm SVE2: up to 2048 bits (64×32); width depends on the processor.

One edge; parallel samples

	r_0	r_1	r_2	r_3	
$v = 2$					...
$v = 4$					...
$v = 7$	lane 0	lane 1	lane 2	lane 3	...
$v = 9$					...

Same row; contiguous state accesses

1 sequential read; 1 sequential write

CPU: vector instructions

($\sim 2 \times$ latency, $> 4 \times$ throughput)

GPU: no divergence in the masked update

Contributing work varies across lanes

Random decisions tied to identity

We changed the execution order.

How do we preserve the same sampled graphs?

Execution order A

1. $(a, 1) \rightarrow 0.20 \rightarrow \text{active}$

2. $(b, 1) \rightarrow 0.80 \rightarrow \text{inactive}$

Recover from identity

$$u(e, r) = H(e, r) \in [0, 1)$$

$$e \in G_r \iff u(e, r) < p_e$$

Example: $p_a = p_b = 0.5$
Same generator seed in both orders

Execution order B

1. $(b, 1) \rightarrow 0.80 \rightarrow \text{inactive}$

2. $(a, 1) \rightarrow 0.20 \rightarrow \text{active}$

(e, r) identifies the edge and sample; values shown are illustrative.

Reconstruct edge membership when needed, without storing each sampled graph.

Compute more, move less

Additional work

- Reconstruct decisions
- Compare probabilities
- Mask inactive sample lanes

Avoided work

- Materializing sampled graphs
- Repeated access to shared topology

~3–21×

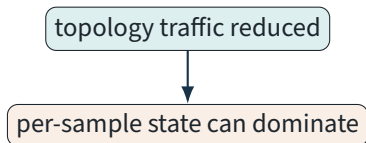
Reported contribution of fused sampling

Sequential FUSED SAMPLING vs. MIXGREEDY;
the three graphs where the reference
completed.

Instruction count can increase while runtime decreases.

InFuseR-MG, Table 4 and accompanying discussion; $K = 50$. On-the-fly sampling only: samples still run separately; batching and memoization are additional changes.

The next bottleneck: per-sample state



Even one exact label per vertex per sample requires:

$$\text{label storage} = \Theta(|V|R)$$

Preserve the information the query needs – not necessarily every intermediate state.

What does the downstream computation actually need?

- exact reached set?
- only its cardinality?

From regularity across samples to uniform state and updates.

Compact representations, uniform updates

Query

How many distinct vertices are reachable?

Count-distinct summary

Hash vertex identities; retain a small register summarizing rare bit patterns.

Why it works: repeated paths do not double-count a vertex; merging the same summary again leaves it unchanged.

Merge through an active edge

$$M_u[j] \leftarrow \max(M_u[j], M_v[j])$$

Fixed-width registers across samples
Contiguous loads and vector maximum

Retain a cardinality estimate instead of materializing every reached set.

HyperFuseR, Secs. 2.2 and 3. One small register per vertex per sample; approximation and error-adaptive rebuilding remain part of the method.

Mapping the sample dimension to hardware

The regular sample dimension stays the same; the low-level mapping changes.

CPU

SIMD lanes

contiguous vector state
cache reuse · masked lanes

GPU

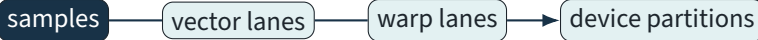
Warp threads

nearby sample-state accesses
similar work within a warp

Multiple GPUs

Sample partitions

local propagation
edge overlap · global reduction



The abstraction is portable; the mapping is architecture-specific.

Research progression: InFuseR-MG → HyperFuseR → DiFuseR.

FASST: sample order changes hardware utilization

Naïve assignment



mixed active / inactive lanes

Sampling-aware assignment



early exit / high utilization

FASST sorts the existing sample keys, then assigns nearby keys to warps and device partitions. For a fixed edge, related decisions can cluster.

A whole inactive warp can skip an edge; a device can omit edges unused by its samples.

DiFuseR, Sec. 4.1. Schematic for one edge; perfect grouping is not guaranteed for every edge.

Sample-space decomposition scales across GPUs

A100 GPUs	Speedup vs. 1 GPU
1	1.00×
2	2.45×
4	4.28×
8	5.64×

Local work

Propagate sketches over each device's samples and required edges.

Global work

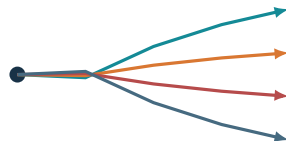
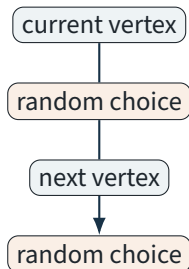
Reduce local estimates to select the next source.

The sample dimension provides a distributed decomposition with useful locality.

DiFuseR, Table 8: geometric means, $K = 50$, $J = 1024$; two GPUs per node for multi-GPU runs. Friendster excluded because the one-GPU configuration does not fit.

A harder case: random walks destroy exact alignment

A random walk is sequential within a trajectory:



trajectories rapidly separate

- next address depends on previous random choice
- exact lane alignment disappears quickly

Now we need a scheduling method, not just a different loop order.

SABA: seed ordering creates opportunities for reuse

First step from s : four equal-probability neighbors.

$[0, .25) \rightarrow A$ $[.25, .50) \rightarrow B$ $[.50, .75) \rightarrow C$ $[.75, 1) \rightarrow D$

Unordered groups of four

seeds	.83	.14	.63	.29
next vertex	D	A	C	B
seeds	.91	.48	.05	.72
next vertex	D	B	A	C

4 distinct adjacency rows per group

Sorted groups of four

seeds	.05	.14	.29	.48
next vertex	A	A	B	B
seeds	.63	.72	.83	.91
next vertex	C	C	D	D

2 distinct adjacency rows per group

Nearby seeds reuse early adjacency; walks can diverge at the next step.

Toy example with normalized, independently drawn seeds; not measured data.

Correct sampling and useful grouping

At a vertex with d neighbors: $x \sim \text{Uniform}\{0, \dots, R-1\}$

$$q = \lfloor R/d \rfloor, \quad k = \lfloor x/q \rfloor, \quad x' = x \bmod q, \quad R' = q$$

Example: $R = 12, d = 3$	$x = 0, \dots, 3$	$x = 4, \dots, 7$	$x = 8, \dots, 11$
--------------------------	-------------------	-------------------	--------------------

Chosen neighbor k	0	1	2
---------------------	---	---	---

Retained residual x'	0, 1, 2, 3	0, 1, 2, 3	0, 1, 2, 3
------------------------	------------	------------	------------

If $q = 0$ or $x \geq dq$: switch permanently to fresh, unbiased conventional sampling.

Uniform next choice given the path history; unused randomness retained for later steps.

Revised SABA manuscript, Sec. 3.2. Assumes ideal independent random inputs. Reordering a fixed set of valid walks is a separate estimator invariance.

Parallelism without locality can increase cache misses

Reported method	L3 misses / naïve
Naïve scalar walks	100.00%
Direct AVX2	108.83%
HASH vector baseline	61.92%
SABA ordering	3.55%

Architectural observation

More vector execution did not ensure fewer cache misses.

Reported ordering comparison

3.38 $\times$ walk-processing speedup over HASH at 16,384 walks/vertex.

Useful parallel execution depends on locality as well as arithmetic throughput.

Scope: historical sampler measurements; these tables do not validate the corrected residual-range method.

SABA manuscript, Sec. 5 validation status and Tables 5, 7. Cache ratios: geometric means over 10 graphs, 16,384 walks/vertex; Broadwell Xeons.

Runtime comparison uses 16 threads.

A methodology for irregular computation

Obstacle	Research move	Work
Repeated topology access	Parallelize across samples; reconstruct decisions	InFuseR-MG
Large intermediate state	Retain mergeable cardinality information	HyperFuseR
Inactive lanes, edge overlap	Order and partition sample space	DiFuseR / FASST
Path-dependent addresses	Group trajectories using sampling structure	SABA

Identify the bottleneck, expose usable regularity, and preserve the relevant contract.

When restructuring pays off

Favorable conditions

- + abundant independent work
- + repeatedly visited sparse structure
- + memory-bound execution
- + compact state / useful summaries
- + schedules that create locality or coherence

Less favorable conditions

- too little work to fill wide machines
- compute-bound kernels
- long trajectories with little reuse
- state that cannot be compressed
- ordering overhead exceeds saved traffic

Saved memory stalls and idle-lane work must exceed ordering, state, and coordination costs.

Ongoing work: fused walks for graph learning

NeuralBloom: accelerating NeuralWalker's input construction

Reduce data movement: fuse walk transitions and structural encodings.

Separate tensor operations

- Dispatch work at successive walk positions
- Move intermediate state through GPU memory
- Construct structural encodings afterward

~2×

Preliminary speedup
in walk computation

Fused GPU execution

One thread advances a walk; private state and encoding construction stay in one kernel.

Ongoing NeuralBloom work. Preliminary compute result; not an end-to-end training speedup.

Ongoing work: reconstruction and trajectory execution

Battus: diffusion reconstruction

Infer missing graph states between sparse snapshots.

- Approximate per-vertex probabilities
- Forward-backward inference
- Regular graph passes mapped to CUDA

Research question

Which inference stages and state are needed for useful reconstruction?

Trajectory-based graph traversal

Represent paths by coordinates that evolve at each step.

- Deterministic trajectory prototype
- Interleave trajectories on the CPU
- Map trajectories to GPU threads

Research question

Can interleaving hide memory latency when locality is limited?

Ongoing projects. Battus uses approximate inference; deterministic trajectories are not claimed to be independent Monte Carlo walks.

Finding regularity systematically

Can we identify useful regularity and decide when exploiting it pays off?

First project: an adaptive trajectory scheduler using observations of reuse and stalls.

Observe	repeated vertices, active lanes, state footprint, scheduling overhead
Decide	keep the order, regroup a batch, or interleave trajectories
Preserve	application-defined dependencies, random streams, and accuracy targets
Deliver	a runtime policy that reduces total execution time on held-out workloads

First workloads: graph walks and NeuralBloom.

Finding Regularity

- Sample structure** Share topology across independent samples.
- Compact representations** Retain useful information; use uniform merges.
- Sampling-aware schedules** Encourage reuse when trajectories diverge.

Next: make these choices systematic, with correctness and cost explicit.

Questions

Backup slides

Backup: fused-sampling implementation in InFuseR-MG

For undirected graphs, use a symmetric edge hash:

$$h(u, v) = \text{MURMUR3}(\min(u, v) \parallel \max(u, v))$$

so $h(u, v) = h(v, u)$.

For simulation r :

$$\rho(u, v)_r = \frac{X_r \oplus h(u, v)}{h_{\max}}$$

and edge (u, v) is active iff

$$\rho(u, v)_r \leq w_{u,v}.$$

The system can reconstruct membership from edge/sample identity without materializing sampled graphs.

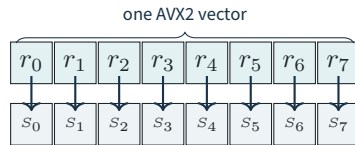
InFuseR-MG, Sec. 3.1. Reconstructibility does not by itself prove joint independence across edges.

Backup: SIMD implementation details

AVX2 processes eight 32-bit sample states at a time.

1. compare component labels
2. broadcast precomputed edge hash
3. XOR with simulation seeds
4. compare against probability threshold
5. masked / blended updates
6. movemask-style reduction for changed lanes

Source: InFuseR-MG, Secs. 3.2–3.3.



Per-vertex sample state is contiguous, so one vector load fetches several simulations for the same vertex.

Backup: HyperFuseR sketch details

Flajolet–Martin style register update:

$$M[j] = \max(M[j], \text{clz}(h_j(x)))$$

Sketch merge:

$$M_{uv}[j] = \max(M_u[j], M_v[j])$$

M_u	2	5	1	4	3	6	2	5
M_v	3	4	6	4	7	2	2	8
max	3	5	6	4	7	6	2	8

Compact registers estimate cardinality; they do not preserve exact reached identities.

Backup: error-adaptive rebuilding in HyperFuseR

After selecting more sources, retained sketches can become less useful for estimating marginal reach.

1. Estimate a candidate's marginal gain from compact state.
2. Compare that estimate with a Monte Carlo evaluation.
3. Rebuild when local and global discrepancy criteria trigger.

The paper uses local threshold $\epsilon_l = 0.3$ and global threshold $\epsilon_g = 0.01$ in its experiments.

These empirical rebuilding thresholds are not universal error guarantees.

Backup: FASST – sample-space tasking

Naïve order

$\{1, 3\}$ $\{2, 0\}$ $\{0, 1\}$ $\{2, 3\}$

Sort / cluster sample keys

$\{0, 1\}$ $\{0, 1\}$ $\{2, 3\}$ $\{2, 3\}$

Principle: exploit exchangeability of independent samples; change assignment and schedule, not the sample set or estimator.

Source: DiFuseR, Sec. 4.1.

On multiple GPUs, each device receives a sample-space partition and derives the local edge set induced by those samples.



Backup: multi-GPU scaling

GPUs	Observed speedup	Ideal
1	1.00×	1×
2	2.45×	2×
4	4.28×	4×
8	5.64×	8×

Interpretation: sample-space decomposition exposes a natural distributed axis with limited synchronization; the claim is not perfect scaling.

DiFuseR, Table 8. Geometric means; $K = 50$, $J = 1024$; two A100 GPUs per node for multi-GPU runs. Friendster excluded.

Backup: performance comparisons and their scope

Result	Reported value	What it demonstrates
InFuseR-MG vs. state of the art	2.3–173.8×	overall effectiveness
Fused sampling contribution	~3–21×	isolated value of fusing
HyperFuseR	up to 119×	compact-state approach
DiFuseR, 1 × A100	3.15 × geom. mean vs. gIM	GPU mapping
DiFuseR, 8 GPUs	5.64 × geom. mean vs. 1 GPU	sample-space scale-out
Historical SABA, 16,384 walks/vertex	6.76 × geom. mean vs. naïve	locality-aware vectorized walks
Historical SABA vs. matched hash base-line	3.38×	scheduling contribution

Interpretation rule

SABA rows describe the earlier sampler and do not validate the corrected residual-range method. Other rows compare complete methods unless explicitly identified as fusion or scaling.

Backup: residual-range sampler and unbiased fallback

Initial seed: $X_W \sim \text{Uniform}\{0, \dots, 2^{64} - 2\}$; set $x = X_W, R = 2^{64} - 1$.

$$q = \lfloor R/d \rfloor, \quad \text{if } q > 0 \text{ and } x < dq: \quad k = \lfloor x/q \rfloor, \quad x \leftarrow x \bmod q, \quad R \leftarrow q.$$

Otherwise switch permanently to conventional sampling. For $H = 2^{32}$ and $1 \leq d < H$:

$$t = H \bmod d, \quad z \sim \text{Uniform}\{0, \dots, H - 1\}, \quad p = zd.$$

Reject and redraw while $p \bmod H < t$; otherwise choose $k = \lfloor p/H \rfloor$.

Proof idea: in residual mode each pair (k, r) has one preimage $x = kq + r$. In fallback, each neighbor has equally many accepted inputs. Both modes preserve the conditional transition law.

Assumptions: independent uniform seeds and fresh independent fallback outputs.

Revised SABA, Sec. 3.2. Degree-zero vertices terminate; practical PRNGs approximate the ideal-input model.

Backup: sampler validity, ordering, and replay

1. **Sampler validity**

Uniform next neighbor conditional on the labeled walk's path history.

2. **Estimator invariance**

For fixed walks, $\frac{1}{N} \sum_j X(W_j) = \frac{1}{N} \sum_j X(W_{\pi(j)})$ in exact arithmetic.

3. **Exact replay across schedules**

Requires initial seeds and fallback streams owned by walk identity.

The manuscript's implementation initializes fallback PRNG states per lane after sorting; a schedule change can change a labeled walk's realized path.

Distributional validity does not imply identical realized samples or bitwise-identical reductions.

Revised SABA, Sec. 3.2. Sorted lane ranks are dependent; the uniformity theorem is about labeled walks.

Backup: SABA locality diagnostic

For bouquet width B , effective pool size β , and distinct items η , an independent-uniform occupancy approximation gives:

$$\frac{\mathbb{E}[\eta]}{B} = \frac{\beta}{B} \left(1 - \left(1 - \frac{1}{\beta} \right)^B \right)$$

approximately

$$\frac{\mathbb{E}[\eta]}{B} \approx \frac{\beta}{B} \left(1 - e^{-B/\beta} \right).$$

How to use it

A trend diagnostic for within-bouquet occupancy and potential reuse – not a bound for the actual state-dependent, seed-sorted walks.

Backup: selected work behind the seminar

1. **Boosting Parallel Influence-Maximization Kernels for Undirected Networks with Fusing and Vectorization**
fused sampling, SIMD, multicore execution
2. **Fast and Error-Adaptive Influence Maximization based on Count-Distinct Sketches**
compact state, sketch propagation, adaptive rebuilding
3. **DiFuseR: A Distributed Sketch-based Influence Maximization Algorithm for GPUs**
GPU sample-space execution, FASST, multi-GPU decomposition
4. **Accelerating Spanning-Centrality Approximation with Sampling-Aware Vectorized Random Walks**
SABA, trajectory scheduling, locality-aware vectorization

Local source PDFs and revised SABA manuscript are listed in [papers/README.md](#).

Backup: SABA evidence provenance

Corrected construction

Residual-range sampling with permanent unbiased fallback; conditional-uniformity argument in Sec. 3.2.

Historical comparisons

Sec. 5 explicitly identifies the performance tables as measurements of the earlier, invalid sampler. The reported 108.83%, 3.55%, and $3.38\times$ figures retain that scope.

Interpretation

Cache behavior of an earlier implementation is not evidence of performance at the corrected path distribution. Regression checks do not replace comparative benchmark measurements.

Revised manuscript: opening of Sec. 5, “Validation status after sampler correction.”

Backup: current projects and their contracts

NeuralBloom

Fused GPU construction of NeuralWalker's walk indices, edge indices, and structural encodings. Preserve the model interface and intended sampling semantics.

Battus

Mean-field forward-backward inference and deterministic decoding. Assess reconstruction quality and graph-causal feasibility separately; this changes the approximation.

Trajectory prototype

Deterministic starting coordinates with CPU interleaving and a Metal GPU implementation. Agreement between implementations is distinct from agreement with an independent random-walk distribution.

Ongoing work; source descriptions in the NeuralBloom, Battus, and randomwalk repositories.

Backup: trajectory coordinates and interleaving

The current deterministic prototype initializes walk i with $x_i = i/N$. At a vertex of degree $d > 0$:

$$k = \lfloor dx \rfloor, \quad x \leftarrow dx - k, \quad v \leftarrow \text{neighbor}(v, k).$$

- CPU: interleave four independent trajectory state chains.
- GPU: one trajectory per thread; exact rational-coordinate arithmetic in Metal.
- Evaluate endpoint approximation separately from execution speed.

Deterministic trajectory rules require a different argument from IID Monte Carlo sampling.

randomwalk/walk.h, metal.h, and AGENTS.md. This prototype does not replenish coordinate precision with fresh randomness.